

Sicurezza e Modello Dati

Crittografia, API Keys, Sandboxing e Permessi

• Crittografia, API Keys, Sandboxing e Permessi

Categoria: Sicurezza e Architettura Dati **Importanza:** ★★★★★

[NOTE] Questo paper descrive il modello di sicurezza e la struttura dati di WhyCremisi. È complementare al Paper 05 (Protocollo) e al Paper 02 (Architettura). Le tecnologie menzionate sono soggette a modifica in base ai requisiti finali di deployment.

• 1. Panoramica della Sicurezza

WhyCremisi opera in un ambiente ibrido: processi locali (plugin bridge, DAW), rete locale (OSC), e potenziali connessioni cloud (AI provider). Ogni superficie espone vettori d'attacco specifici.

◆ PROCESS FLOW

THREAT MODEL — SUPERFICI

MACHINE LOCALE RETE LOCALE

- Accesso al disco • Intercettazione
- Lettura API key pacchetti OSC
- Process injection • WS hijacking
- File di log • MITM su localhost

CLOUD AI API APPLICAZIONE

- Key leakage • Session hijack
- Prompt injection • Comandi non
- Data exfiltr. autorizzati DAW
- Provider breach • Plugin malevolo

Approccio: Defense-in-Depth. Nessun singolo meccanismo è sufficiente. La sicurezza è stratificata su 5 livelli:

LIVELLO 5 – Audit & Monitoring	(rilevamento)
LIVELLO 4 – Applicazione	(sandboxing, permessi)
LIVELLO 3 – Comunicazione	(TLS, WSS, crittografia)
LIVELLO 2 – Archiviazione	(AES-256-GCM, keychain)
LIVELLO 1 – Fisico/Sistema	(S0, accesso utente)

VETTORE	IMPATTO	MITIGAZIONE
Key leak	Critico	Keychain + env vars + rotation
MITM rete	Alto	TLS 1.3 + WSS
Plugin malevolo	Alto	Sandboxing + timeout + rollback
Accesso disco	Medio	AES encryption at rest
Session hijack	Medio	Token per-sessione + expiry
Log exposure	Basso	Redazione automatica secrets

• 2. Gestione delle API Keys

Le API keys sono il punto di accesso ai provider AI (Gemini, OpenAI, Anthropic). Una loro esposizione equivale a perdita di controllo dell'agente.

◆ OVERVIEW

API KEY LIFECYCLE

INPUT → OBFUSCATION → ENCRYPTION → KEYCHAIN → USO

(runtime) (AES-256) (macOS)

Ogni key passa attraverso 4 fasi prima di essere usata.

Mai in chiaro su disco. Mai nei log. Mai in git.

— 2.1 INTEGRAZIONE KEYCHAIN (MACOS)

Le API keys sono memorizzate nel **macOS Keychain** (Servizio `whycremisi-keys`), non in file di configurazione.

```
// APIKeyManager.cpp - pseudocodice
class APIKeyManager {
    SecKeychainRef keychain;

    void storeKey(const std::string& provider, const std::string& key) {
        // SecKeychainAddGenericPassword con ACL processo-only
        // Attributi: provider="gemini|ollama|openai|anthropic"
        // Accessibile solo dal processo WhyCremisi
    }

    std::string retrieveKey(const std::string& provider) {
        // 1. Prova keychain
        // 2. Fallback: environment variable WHYCREMISI_KEY_<PROVIDER>
        // 3. Fallback: encrypted config file in ~/.whycremisi/config.enc
        // MAI: loggare la chiave
    }

    void rotateKey(const std::string& provider, const std::string& newKey) {
        // Rotazione a caldo senza riavvio
        // Vecchia chiave mantenuta per 60s per richieste in-flight
    }
};
```

— 2.2 CIFRATURA A RIPOSO (AT REST)

Per sistemi senza keychain (Linux, Windows), le chiavi sono cifrate in un file di configurazione:

◆ OVERVIEW

~/.whycremisi/

config.enc → AES-256-GCM cifrato

config.enc.salt → Sale PBKDF2 (16 byte)

config.enc.nonce → Nonce per AES-GCM (12 byte)

Deriva chiave: PBKDF2-HMAC-SHA256, 600.000 iterazioni

Cifatura: AES-256-GCM (autenticato, con tag 16 byte)

Sblocco: Chiave derivata da password utente + salt

— 2.3 OBFUSCATION IN MEMORIA

In runtime, la chiave non deve rimanere in chiaro in memoria più del necessario:

```
// SecureString - RAII wrapper con zeroizzazione
class SecureString {
    std::unique_ptr<char[]> data;
    size_t length;

    ~SecureString() {
        // volatile write per prevenire ottimizzazione compilatore
        volatile char* p = data.get();
        for (size_t i = 0; i < length; i++) p[i] = 0;
    }
};
```

— 2.4 ROTAZIONE DELLE CHIAVI

◆ OVERVIEW

- Rotazione manuale: tramite UI (Impostazioni → API Keys)
- Rotazione automatica: su rilevamento uso sospetto
- Grace period: 60s overlap tra old e new key
- Notifica: alert UI se una key sta per scadere

— 2.5 REGOLE DI NON-ARCHIVIAZIONE

DOVE NON VA	PERCHÉ
File <code>.env</code>	Facile commit accidentale in git
Log file	Visibile in console/app di logging
Git history	Permanente, non riscrivibile
Crash dump	Analizzato da terzi
Preferenze DAW	Accessibile da altri plugin
Sessione AI	Inviato al provider come prompt

Regola aurea: se un dato esce dal processo WhyCremisi, non deve contenere API keys in chiaro.

• 3. Crittografia dei Dati

— 3.1 CIFRATURA A RIPOSO (PERSISTENT STORAGE)

Tutti i dati persistenti sensibili usano **AES-256-GCM**:

```
{
  "algoritmo":      "AES-256-GCM",
  "key_size":       256,
  "nonce_size":     12,    // bytes, raccomandato da NIST
  "tag_size":       16,    // GCM authentication tag
  "derivazione":    "PBKDF2-HMAC-SHA256 × 600000 iterazioni",
  "modalità":       "AEAD (cifrazione + autenticazione)"
}
```

Dati cifrati:

- API keys (file config.enc)
- Profili utente (user_profile.json.enc) — [NOTE] selezionabile
- Token di sessione
- Cache di plugin discovery

Dati NON cifrati (performance critical):

- Log di audit (vedi §8)
- Preferenze UI

— 3.2 CRITTOGRAFIA DI RETE

◆ OVERVIEW

PROTOCOLLO CIFRATURA PORTA AUTENTICAZIONE

WebSocket WSS (TLS 1.3) 443 Certificato + token

OSC (rete) TLS 1.3 μTLS 8910 Certificato mutuo

OSC (local) Nessuna 8910 Solo localhost

AI API TLS 1.3 — API key in header

Per OSC su localhost si può optare per **nessuna cifratura** dato che il traffico non esce dalla macchina. Per OSC in rete (multi-macchina), si usa **μTLS** con certificati autofirmati scambiati al pairing.

— 3.3 SCHEMA DI CIFRATURA PER FILE

◆ DATA SCHEMA

Plaintext → AES-256-GCM Enc → File cifrato

(JSON) + nonce + tag .enc

HMAC-SHA256

Integrity

Check

• 4. Sandboxing delle Esecuzioni

WhyCremisi esegue tool calls (comandi DAW, modifica parametri, esecuzione script) che possono avere effetti irreversibili sul progetto musicale. Ogni tool call è sandboxata.

— 4.1 ARCHITETTURA DI ESECUZIONE

◆ OVERVIEW

TOOL EXECUTION SANDBOX

Richiesta AI → Permission Check → Execute → Validate

→ → → →

Type+Target Livello permesso Timeout Rollback?

+Params corrente watchdog su fallim.

— 4.2 TIMEOUTPER TOOL CALL

Ogni tool call ha un timeout rigido:

TIPO TOOL	TIMEOUT DEFAULT	TIMEOUT MASSIMO
<code>daw.command</code> (play/stop)	2s	5s
<code>daw.command</code> (setTempo, setVolume)	1s	3s
<code>plugin.control</code>	500ms	2s
<code>plugin.query</code>	300ms	1s
<code>ai.prompt</code>	15s	60s
Script utente	5s	30s

Se il timeout scatta:

- 1. Il tool viene terminato forzatamente
- 2. Lo stato viene ripristinato al precedente (se possibile)
- 3. L'errore viene loggato nell'audit
- 4. L'utente viene notificato

— 4.3 ROLLBACK SU FALLIMENTO

◆ OVERVIEW

MECCANISMO DI ROLLBACK

1. Snapshot stato → capture parametri PRE modifica

2. Esecuzione tool

3. Se successo → conferma, aggiorna snapshot

4. Se fallimento → ripristino parametri PRE

5. Se rollback fallisce → stato EMERGENZA, alert utente

Snapshot: { trackId, pluginId, paramName, oldValue,

timestamp, contextHash }

— 4.4 LIVELLI DI PERMESSO (TOOL EXECUTION)

LIVELLO	DESCRIZIONE	DEFAULT	RICHIEDE
<code>read-only</code>	Solo query, nessuna modifica	✓	—
<code>suggest</code>	Mostra suggerimento, esecuzione manuale	—	Conferma
<code>auto-execute</code>	Esegue automaticamente se confidenza > 85%	—	Config
<code>full-control</code>	Esegue qualsiasi comando senza conferma	—	Config + avviso

[NOTE] Il livello `full-control` è destinato solo a utenti esperti. La sua attivazione richiede una conferma esplicita con avviso dei rischi. I log di audit includono uno special flag `full_control:true`.

• 5. Modello dei Permessi

— 5.1 MATRICE DEI PERMESSI

La matrice definisce per ogni utente quali azioni sono consentite su quali target:

◆ OVERVIEW

read-only suggest auto-exec full-control

Play/Stop	✓	✓	✓	✓
Trasporto	✓	✓	✓	✓
Volume Traccia	✓	✓	✓	✓
Mute/Solo	✓	✓	✓	✓
EQ Param	✓*	✓	opt-in	✓
Comp Param	✓*	✓	opt-in	✓
FX Insert	✓*	—	—	✓
FX Remove	✓*	—	—	✓
Script Esegui	—	—	opt-in	✓
Master Limiter	✓*	✓	—	✓

* = read-only sui valori attuali

— 5.2 GRANULARITÀ

I permessi sono configurabili a 3 livelli di granularità:

◆ OVERVIEW

LIVELLO 1 — Globale (per azione)

Es: "EQ Param" → suggest per TUTTE le tracce

LIVELLO 2 — Per Traccia/Plugin

Es: "EQ Param → auto-execute su Traccia 1 (Kick)"

"EQ Param → read-only su Traccia 3 (Vocali)"

LIVELLO 3 — Per Parametro Specifico

Es: "Master Limiter.Gain → read-only"

"Master Limiter.TruePeak → suggest"

— 5.3 PER-SESSION OVERRIDE

L'utente può concedere override temporanei per la sessione corrente:

```
{
  "session0verrides": [
    {
      "target": "track:3:plugin:Pro-Q3:param:Band1_Gain",
      "level": "auto-execute",
      "reason": "sto producendo, voglio velocità",
      "expiresAt": 1716503600000
    }
  ]
}
```

— 5.4 AUDIT LOG DELLE AZIONI

Tutte le azioni execute (incluse quelle automatiche) sono registrate:

```
{
  "timestamp": 1716500000000,
  "userId": "whyed",
  "action": "plugin.control",
  "target": "track:1:plugin:Pro-Q3:param:Band1_Gain",
  "oldValue": 0.0,
  "newValue": -4.5,
  "permissionLevel": "auto-execute",
  "confidence": 0.92,
  "status": "success",
  "sessionId": "abc-123"
}
```

• 6. Modello Dati

Struttura dati completa del sistema WhyCremisi.

— 6.1 PLUGIN RECORD

CAMPO	TIPO	DESCRIZIONE
<code>id</code>	UUID v4	Identificatore univoco
<code>name</code>	string	Nome plugin (es. "Pro-Q3")
<code>vendor</code>	string	Produttore (es. "FabFilter")
<code>version</code>	string	Versione plugin
<code>paramCount</code>	int	Numero parametri esposti
<code>category</code>	enum	eq, compressor, reverb, limiter, synth, utility, other
<code>tags</code>	string[]	Tag liberi ("vintage", "transparent", "analog")
<code>knownProcedures</code>	Procedure[]	Procedure note per questo plugin (vedi Paper 03)


```
{
  "id": "a1b2c3d4-...",
  "name": "Pro-Q3",
  "vendor": "FabFilter",
  "version": "3.2.1",
  "paramCount": 48,
  "category": "eq",
  "tags": ["digital", "transparent", "surgical", "linear-phase"],
  "knownProcedures": []
}
```

— 6.2 SESSION RECORD

CAMPO	TIPO	DESCRIZIONE
id	UUID v4	ID univoco sessione
daw	string	Nome DAW ("REAPER", "Logic Pro", etc.)
tracks	Track[]	Array tracce nel progetto
plugins	Plugin[]	Plugin caricati nella sessione
aiState	object	Stato agente all'inizio sessione
createdAt	timestamp	Data/ora creazione
lastActivity	timestamp	Ultima azione registrata

```
{
  "id": "session-001",
  "daw": "REAPER",
  "tracks": [
    {
      "id": 1,
      "name": "Kick",
      "color": "#FF4400",
      "pluginSlots": [
        { "slot": 0, "pluginId": "a1b2c3d4-...", "active": true },
        { "slot": 1, "pluginId": "e5f6g7h8-...", "active": true }
      ]
    }
  ],
  "plugins": [],
  "aiState": {
    "personality": "professional_warm",
    "memoryVersion": 2
  },
  "createdAt": 1716500000000,
  "lastActivity": 1716503600000
}
```

— 6.3 PRESET RECORD

CAMPO	TIPO	DESCRIZIONE
<code>id</code>	UUID v4	Identificatore univoco
<code>name</code>	string	Nome del preset
<code>pluginId</code>	UUID v4	Riferimento al Plugin
<code>params</code>	map<string, float>	Mappa nome-parametro → valore
<code>tags</code>	string[]	Tag di ricerca
<code>createdAt</code>	timestamp	Data creazione
<code>usageCount</code>	int	Numero di utilizzi

```
{
  "id": "preset-001",
  "name": "Kick Tight 808",
  "pluginId": "a1b2c3d4-...",
  "pluginName": "Pro-Q3",
  "params": {
    "Band1_Freq": 55.0,
    "Band1_Gain": 4.0,
    "Band1_Q": 1.2,
    "Band2_Freq": 220.0,
    "Band2_Gain": -3.5,
    "Band2_Q": 8.0
  },
  "tags": ["kick", "tight", "808", "sub-bass"],
  "createdAt": 1716400000000,
  "usageCount": 12
}
```

— 6.4 AUDITENTRY RECORD

CAMPO	TIPO	DESCRIZIONE
<code>timestamp</code>	timestamp	Quando è avvenuta l'azione
<code>userId</code>	string	Identificativo utente
<code>action</code>	string	Tipo azione (es. <code>plugin.control</code>)
<code>target</code>	string	Target specifico
<code>oldValue</code>	any?	Valore precedente (se applicabile)
<code>newValue</code>	any?	Nuovo valore (se applicabile)
<code>status</code>	enum	success, error, warning, timeout, rollback
<code>details</code>	string	Note aggiuntive
<code>permissionLevel</code>	string	Livello permesso usato

— 6.5 RELAZIONI TRA ENTITÀ

◆ OVERVIEW

1N NM

UTENTE SESSIONE PLUGIN

11

→ →

AUDIT PRESET

LOG

PERMESSO

OVERRIDE

• 7. Privacy

WhyCremisi è progettato per funzionare **local-first**. I dati audio non lasciano mai la macchina dell'utente.

— 7.1 PRINCIPI FONDAMENTALI

◆ OVERVIEW

PRINCIPIO IMPLICAZIONE

Local-first AI può funzionare interamente offline

(Ollama). Dati audio: mai in cloud.

Analytics opt-in Telemetria disabilitata per default.

L'utente deve attivarla esplicitamente.

Trasparenza L'utente vede esattamente quali dati

vengono raccolti in ogni momento.

Diritto cancellazione L'utente può cancellare TUTTI i propri

dati in qualsiasi momento.

— 7.2 COSA NON VIENE MAI RACCOLTO

- Campioni audio grezzi
- Nomi di progetto (a meno di analytics opt-in)
- Metadati di file utente
- Contenuto di plugin non WhyCremisi
- Keystrokes o interazioni UI non relative a WhyCremisi

— 7.3 RETENTION POLICY

DATO	RETENTION	CANCELLAZIONE
Log audit	90 giorni	<code>/audit/clear</code> o automatica
Profilo utente	Finché attivo	<code>/reset</code>
Sessione JSONL	30 giorni	Rotazione automatica
API keys	Finché valide	Rotazione manuale
Analytics	12 mesi	Opt-out immediato
Cache plugin	7 giorni	Ricreata al bisogno

— 7.4 DIRITTO ALLA CANCELLAZIONE

LAYER STACK

`/reset` → azzera profilo, sessioni, permessi

`/forget <pattern>` → cancella memoria specifica

`/audit/export` → esporta audit log in JSON

`/audit/clear` → cancella audit log

`/data/export` → esporta tutti i dati utente

`/data/delete` → cancella tutti i dati (GDPR-style)

• 8. Audit Logging

Ogni azione eseguita dal sistema è tracciata in un audit log immutabile (append-only).

— 8.1 STRUTTURA AUDIT ENTRY

```
{
  "timestamp": 1716500000000,
  "type": "plugin.control",
  "userId": "whyed",
  "sessionId": "session-001",
  "target": "track:1:plugin:Pro-Q3:param:Band1_Gain",
  "before": 0.0,
  "after": -4.5,
  "status": "success",
  "permission": "auto-execute",
  "confidence": 0.92,
  "duration_ms": 120,
  "ip": "127.0.0.1",
  "userAgent": "WhyCremisi/2.0",
  "details": "Taglio EQ suggerito dall'agente, accettato automaticamente"
}
```

◆ DATA SCHEMA

~/whycremisi/audit/

audit.log → Current (append)

audit.2025-05-12.json → Rotated daily

audit.2025-05-11.json

audit.2025-05-10.json.gz → Compressed after 7 days

...

- Rotazione: Giornaliera, a mezzanotte
- Compressione: gzip dopo 7 giorni
- Retention: 90 giorni (poi cancellazione automatica)
- Dimensione max: 100MB per file (se superata, split immediato)

— 8.3 EXPORT E VISUALIZZAZIONE

◆ TEST PIPELINE

COMANDO FORMATO

/audit/export JSON array

/audit/export?format=csv CSV

/audit/export?since=7d Solo ultimi 7 giorni

/audit/export?type=error Solo errori

/audit/search?q=Pro-Q3 Ricerca testuale

UI: Pannello Audit in Impostazioni, con:

- Timeline interattiva
- Filtri per tipo/status/data
- Dettaglio a espansione
- Pulsante Export

— 9.1 PRODUCTION HARDENING CHECKLIST

- ❑ TLS 1.3 configurato su tutte le connessioni remote
- ❑ Certificati firmati (Let's Encrypt o internal CA)
- ❑ pTLS per comunicazione multi-macchina
- ❑ API keys in vault / 1Password CLI / Doppler
- ❑ Log rotazione e retention configurati
- ❑ rate limiting sulle API (100 req/min per provider)
- ❑ timeout tool call configurati per ambiente
- ❑ permessi utente impostati (default: suggest)
- ❑ sandboxing attivo e verificato
- ❑ monitoring endpoint /health e /metrics
- ❑ alerting su: error rate > 5%, timeout > 10%, key rotation needed
- ❑ backup cifrato dei config.enc

— 9.2 NETWORK SEGMENTATION

◆ OVERVIEW

TOPOLOGIA DI RETE

VLAN 10

DAW Host WhyCrem

(Studio) OSC/WSS Bridge

→ → →

React UI AI API Ollama

(Browser) (Cloud) (Local)

Tutto il traffico tra DAW e Bridge su VLAN isolata.

AI API solo in uscita su whitelist di dominio.

— 9.3 SECRETS MANAGEMENT

STRUMENTO	USO	VANTAGGIO
macOS Keychain	API keys locali	Nativamente sicuro
1Password CLI	Secrets CI/CD	Audit + rotazione
HashiCorp Vault	Multi-macchina	Dynamic secrets
Mozilla SOPS	GitOps cifrato	Git-friendly encryption
Doppler	Team deployment	Environment inheritance

◆ OVERVIEW

METRICHE CHIAVE (prometheus format)

```
whycremisi_tool_calls_total{status="success|error|timeout"}
whycremisi_api_key_last_rotation{provider="gemini"}
whycremisi_sandbox_timeout_total
whycremisi_audit_entries_total
whycremisi_websocket_connections_active
whycremisi_permission_blocks_total
```

ALERT SOGLIE

- Critical: error rate > 10% su 5 min
- Warning: tempo medio risposta > 2x baseline
- Info: rotazione key necessaria (> 80% validità)

— 9.5 SECURITY REVIEW CADENCE

- ▶ Automated: Ogni PR passa SAST (Semgrep) + dependency audit
- ▶ Manuale: Review trimestrale del threat model
- ▶ Pentest: Annuale su superficie rete e API
- ▶ Bounty: Programma bug bounty (invito) per researcher

• 10. Riepilogo

La sicurezza di WhyCremisi poggia su tre pilastri: **prevenzione** (crittografia, sandboxing, permessi), **rilevamento** (audit logging, monitoring) e **risposta** (rollback, rotation, alerting). Il threat model è pensato per un ambiente di produzione musicale professionale dove il danno potenziale non è un data breach ma la corruzione di un progetto artistico.

◆ LAYER STACK

RIEPILOGO STRATIFICAZIONE

PREVENZIONE RILEVAMENTO RISPOSTA

```
AES-256-GCM Audit log Rollback
TLS 1.3 / WSS Monitoring Key rotation
Sandbox + timeout Alerting Notifica utente
Matrice permessi /audit/export /reset
Keychain + env var Search log
Obfuscation memory
Defense-in-depth
```

[NOTE] La sicurezza è un processo, non uno stato. Questo paper è solo un'istantanea e non una guida.

